Hardware/Software Co-Design for MAYO Verification

Project Proposal

Kowsyap Pranay Kumar Musumuri
G01503265
ECE 615

Motivation

Why Post-Quantum Signatures?

The challenge? Quantum computers.

- They're powerful. They solve problems like **integer factorization** and **discrete logarithms** easily.
- The problem? These are the foundations of today's encryption.

The solution? Post-Quantum Cryptography (PQC).

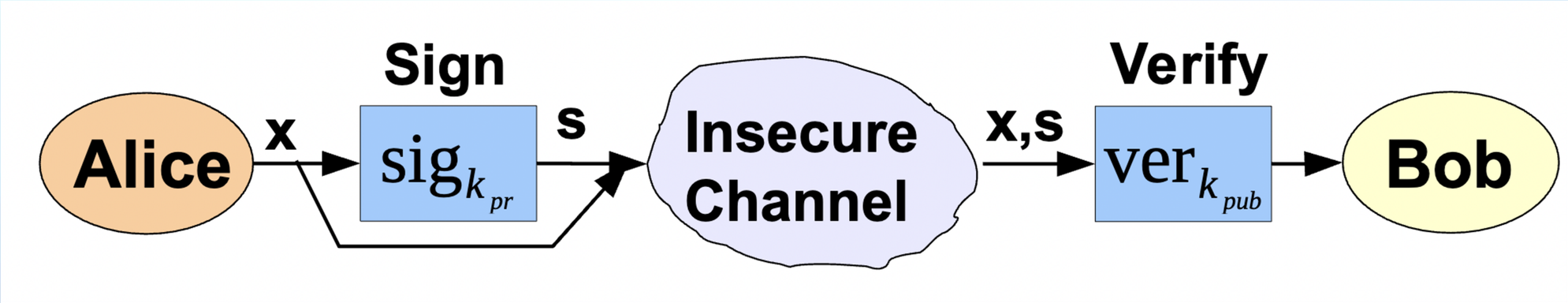
- PQC designs cryptographic systems that stay secure even if quantum computers become common.

MAYO Signature Scheme

What Is MAYO?

Parameter set security level	MAYO_1 1	MAYO_2 1	MAYO_3 3	MAYO_5 5
n	86	81	118	154
m	78	64	108	142
o	8	17	10	12
k	10	4	11	12
q	16	16	16	16
salt_bytes	24	24	32	40
digest_bytes	32	32	48	64
pk_seed_bytes	16	16	16	16
$f(z)$	$f_{78}(z)$	$f_{64}(z)$	$f_{108}(z)$	$f_{142}(z)$
secret key size	24 B	24 B	32 B	40 B
public key size	1420 B	4912 B	2986 B	5554 B
signature size	454 B	186 B	681 B	964 B
expanded sk size	141 KB	99 KB	367 KB	822 KB
expanded pk size	142 KB	104 KB	370 KB	828 KB

- MAYO is a multivariate post-quantum digital signature scheme
- Based on operations over small finite fields, especially GF(16)
- Designed to offer strong security with compact computations
- Signature verification involves structured matrix/vector operations and hashing



Thesis Overview

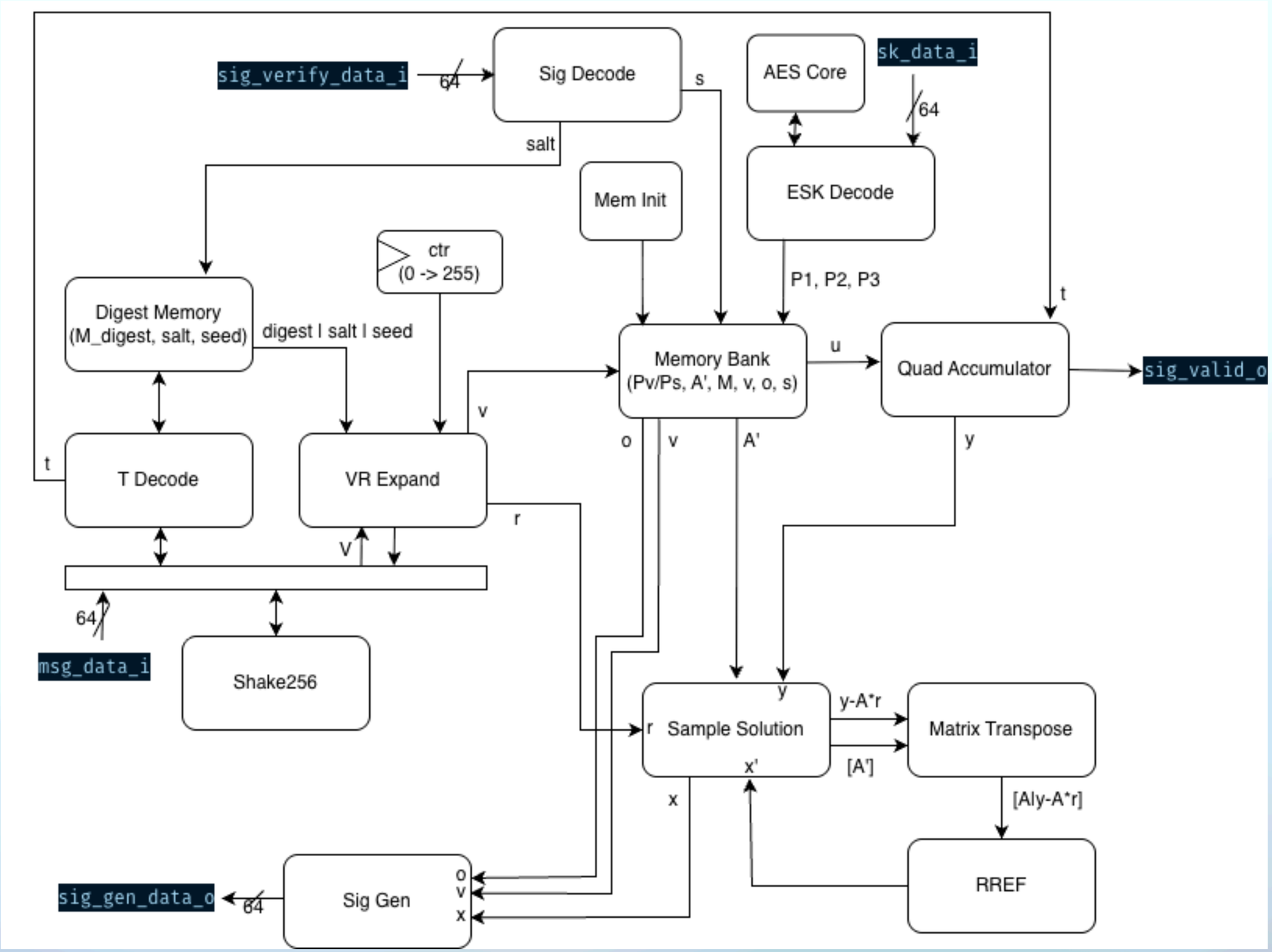
Title: Hardware Implementation of MAYO Signature Algorithm

Goal: Implement high-performance hardware architecture for post-quantum signature MAYO with all security levels (1,3,5) for round 1 and 2.

Tools: VHDL, Vivado

Advisor: Prof. Gaj

Level	Round 1				Round 2			
	Signing (cc / ms)	Verify (cc / ms)	Hardware(LUT/FF/BR)	Clock (ns)	Signing (cc / ms)	Verify (cc / ms)	Hardware(LUT/FF/BR)	Clock (ns)
1	**9.9k / 0.11**	**3.3k / 0.04**	118k/47k/79.5	10.2	14.3k / 0.15	5.2k / 0.06	169k/63k/98.5	10.2
1	11.6k / 0.11	3.7k / 0.04	96k/43k/95.5	8.8	**11.9k / 0.11**	**4k / 0.04**	92k/43k/91	8.8
3	**20.1k / 0.23**	**6.8k / 0.08**	205k/76k/132.5	11	**25.5k / 0.29**	**9.1k / 0.11**	248k/87k/143.5	11
5	**34k / 36.7**	**11.4k / 0.13**	302k/104k/194.5	10.8	**41.8k / 0.48**	**14.8k / 0.16**	305k/118k/206.5	10.8


sig_verify_data_i
64
Sig Decode
s
AES Core
sk_data_i
64
salt
Mem Init
ESK Decode
ctr
(0 -> 255)
P1, P2, P3
t
Digest Memory
(M_digest, salt, seed)
digest | salt | seed
Memory Bank
(Pv/Ps, A', M, v, o, s)
u
Quad Accumulator
sig_valid_o
v
o
v
A'
y
t
T Decode
VR Expand
r
V
64
msg_data_i
Shake256
y
y-A*r
r
Sample Solution
Matrix Transpose
x'
[A']
x
[Aly-A*r]
RREF
sig_gen_data_o
64
Sig Gen
o
v
x

Verification Algorithm

Algorithm 6 MAYO.ExpandPK(cpk)

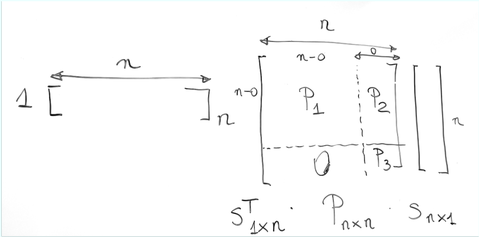
Input: Compact public key cpk $\in \mathcal{B}^{\text{cpk_bytes}}$
Output: Expanded public key epk $\in \mathcal{B}^{\text{epk_bytes}}$

```
//Parse cpk.
seed_pk ← cpk[0 : pk_seed_bytes]

//Expand seed_pk and return.
epk = AES-128-CTR(seed_pk, P1_bytes + P2_bytes) || cpk[pk_seed_bytes : pk_seed_bytes + P3_bytes]
return epk.
```

Inputs: message, signature, public key

Output: valid / invalid



Main steps:

- decode **signature**-related values
- derive **target** (***t***) value using SHAKE256
- expand **public-key**-related data using AES128 and then decode the data into m matrices of n x n
- compute verification-side **GF(16) matrix/vector operations** of n x n matrices, n = 81
- **compare** computed result with target

Algorithm 8 MAYO.Verify(epk, M, sig)

Input: Expanded public key epk $\in \mathcal{B}^{\text{epk_bytes}}$
Input: Message M $\in \mathcal{B}^*$
Input: Signature sig $\in \mathcal{B}^{\text{sig_bytes}}$
Constant: $\mathbf{E} \in \mathbb{F}_q^{m \times m}$ // Represents multiplication by z in $\mathbb{F}_q[z]/(f(z))$
Output: An integer result to indicate if sig is valid (result $= 0$) or invalid (result < 0).

```
//Decode epk.
P1_bytestring ← epk[0 : P1_bytes]
P2_bytestring ← epk[P1_bytes : P1_bytes + P2_bytes]
P3_bytestring ← epk[P1_bytes + P2_bytes : P1_bytes + P2_bytes + P3_bytes]
{P_i^(1)}_{i∈[m]} ← Decode_P(1)(P1_bytestring)      // P_i^(1) ∈ F_q^((n−o)×(n−o)) upper triangular
{P_i^(2)}_{i∈[m]} ← Decode_P(2)(P2_bytestring)      // P_i^(2) ∈ F_q^((n−o)×o)
{P_i^(3)}_{i∈[m]} ← Decode_P(3)(P3_bytestring)      // P_i^(3) ∈ F_q^(o×o) upper triangular

//Decode sig.
salt ← sig[⌈nk/2⌉ : ⌈nk/2⌉ + salt_bytes]
s ← Decode_vec(kn, sig)
for i from 0 to k − 1 do
    s_i ← s[i ∗ n : (i + 1) ∗ n]

//Hash message and derive t.
M_digest ← SHAKE256(M, digest_bytes)                // M_digest ∈ B^digest_bytes
t ← Decode_vec(m, SHAKE256(M_digest || salt, ⌈m log(q)/8⌉))   // t ∈ F_q^m

//Compute P*(s).
y ← 0_m                                             // y ∈ F_q^m
ℓ ← 0
for i from 0 to k − 1 do
    for j from k − 1 to i do
        u ← { { s_i^T (P_a^(1)  P_a^(2); 0  P_a^(3)) s_i }_{a∈[m]}                                  if i = j
              { s_i^T (P_a^(1)  P_a^(2); 0  P_a^(3)) s_j + s_j^T (P_a^(1)  P_a^(2); 0  P_a^(3)) s_i }_{a∈[m]}   if i ≠ j   // u ∈ F_q^m
        y ← y + E^ℓ u
        ℓ ← ℓ + 1

//Accept signature if y = t.
if y = t then
    return 0
return −1
```

Proposed HW/SW Partition

Software Tasks vs Hardware Tasks

Software on PS

- MAYO verify control flow
- signature decoding
- target generation using SHAKE256
- public-key expansion using AES-128
- Send the decoded data to PL using DMA

Hardware on PL

- reduced GF(16) verification kernel
- data_storage(BRAMS) to store matrices data
- Hardware accelerators for GF(16) matrix multiplications, vector accumulations, and reduction mod over $f_{64}(z)$
- final comparison and valid/invalid generation

$$f_{64}(z) = z^{64} + x^3 z^3 + x z^2 + x^3$$

Conclusion

- Area-efficient MAYO verify implementation for Z7-10
- MAYO verification is a suitable candidate for HW/SW co-design
- Software will handle control, decoding, AES, and Keccak/SHAKE
- Hardware will accelerate the core GF(16) verification datapath
- The proposed design aims to balance resource usage, latency, and practicality for resource-constrained FPGA-SoC platforms

Thank you